



REAL-TIME FOOD DELIVERY STREAM ANALYTICS

Team Briefing · Spring 2026

IE University · Stream Analytics Group Project

1. What Is This Project?

We are building a real-time analytics pipeline that simulates the data infrastructure behind an on-demand food delivery platform — think Uber Eats or Glovo. The platform continuously generates thousands of events per minute: orders being placed, restaurants cooking, couriers picking up meals, and deliveries being completed. Our job is to capture, process, and visualise that stream of data as it happens.

This is not a batch analytics project. Everything runs in motion — data flows continuously, and our pipeline must react within seconds. That is what makes stream processing fundamentally different from traditional data engineering.

The Pipeline at a Glance

① Generate	② Ingest	③ Process	④ Store	⑤ Visualise
Synthetic events via Python generator	Azure Event Hubs (Kafka-compatible broker)	Spark Structured Streaming with windowed SQL	Parquet files in Azure Blob Storage	Live Streamlit dashboard

The project is split into two graded milestones. Milestone 1 is about designing and generating the data. Milestone 2 is about processing and visualising it. This document focuses on Milestone 1 and how we will divide the work.

2. Our Two Data Feeds

We designed two complementary event feeds that together cover the full operational lifecycle of a food delivery order.

Feed 1 — Order Lifecycle Events
Purpose: Tracks every state transition an order goes through — from placement to delivery or cancellation. This is the central transaction log of the platform. Flow: <i>PLACED</i> → <i>ACCEPTED</i> → <i>PREPARING</i> → <i>READY_FOR_PICKUP</i> → <i>PICKED_UP</i> → <i>DELIVERED</i> (or <i>CANCELLED</i> at any stage) Key fields: <i>order_id</i>, <i>event_type</i>, <i>event_time</i> (ms), <i>courier_id</i>, <i>restaurant_id</i>, <i>zone_id</i>, <i>order_value_eur</i>, <i>device_id</i>, <i>promo_applied</i> Enables: Delivery time KPIs, cancellation rate windows, fraud detection, zone-level demand surge indicators

Feed 2 — Restaurant Events

Purpose: Captures kitchen-side operational milestones: when orders are received, prep starts, prep ends, and whether SLA thresholds were breached.

Flow: *ORDER_RECEIVED* → *PREP_STARTED* → *PREP_COMPLETE* → *HANDOFF_TO_COURIER* (+ *SLA_BREACH*, *RESTAURANT_BUSY / AVAILABLE*)

Key fields: *order_id* (join key), *restaurant_id*, *event_type*, *event_time* (ms), *prep_duration_seconds*, *sla_threshold_seconds*, *queue_depth*, *capacity_pct*, *cuisine_type*

Enables: Prep-time SLA monitoring, kitchen overload alerts, demand-supply health per zone (joined with Feed 1)

The two feeds are joined on *order_id*, which means every kitchen milestone can be correlated with the customer-facing order state. This join is what unlocks the advanced use cases in Milestone 2 (anomaly detection, surge prediction).

3. Milestone 1 — What We Are Building

Milestone 1 has four graded sub-sections. Each one has a concrete deliverable that will end up in the GitHub repository. The descriptions below tell each teammate exactly what they need to produce and why it matters.

1.1 — Create Two Feeds

We must justify our choice of feeds in the documentation — not just describe them. The grader wants to see: why these two feeds are essential, what analytics they enable, and how their schemas support event-time processing and late data handling.

- Feed 1 — Order Lifecycle Events: every state transition from PLACED to DELIVERED or CANCELLED
- Feed 2 — Restaurant Events: kitchen milestones from ORDER_RECEIVED through PREP_COMPLETE and SLA_BREACH
- Both feeds share *order_id* as a join key, enabling stream-table joins later

1.2 — Schema & Formats

For each feed we need a valid AVRO schema (.avsc file) plus actual event output in both JSON and AVRO binary format.

- AVRO schemas must use correct types, enums, nullable unions (["null", type]), and a *schema_version* field
- *event_time* (epoch ms) is mandatory on every event — this is what Spark will use for windowing in M2

- Join identifiers (order_id, restaurant_id, zone_id) must be present in both feeds
- All schemas are already implemented in schemas/ — review them and flag missing fields before we generate at scale

1.3 — Realism Requirements

This is the most technically demanding sub-section. The generator must produce data realistic enough to stress-test a real stream processor.

- Realistic distributions: lunch peak (12-14h), dinner peak (19-22h), weekend multiplier, zone-level demand skew
- Configurability: restaurants, couriers, cancellation probability, surge probability, promo periods — all via CLI flags
- Edge cases (non-negotiable for the grade):
- Out-of-order events — 5% of events delayed up to 8 min to simulate late arrivals
- Duplicate events — 2% re-emitted with a new event_id to test deduplication logic
- Missing steps — 1% of orders go DELIVERED without a prior PICKED_UP
- Impossible durations — 0.5% of prep events exceed 3600 s to trigger anomaly detection
- Courier going offline mid-delivery — 2% end with CANCELLED / COURIER_TIMEOUT

1.4 — Deliverables (GitHub Repository)

Everything below must be in the private GitHub repo by the submission deadline:

- README.md — project overview, team structure, design note with field descriptions, assumptions, planned analytics
- generator/ — Python code with its own README, CLI reference, and requirements.txt
- schemas/ — order_lifecycle.avsc and restaurant_events.avsc
- sample_data/ — small JSON and AVRO batches for both feeds (already generated: 481 + 396 events)

Milestone 2 (coming later): Once M1 is submitted, we move to stream ingestion (Azure Event Hubs), Spark Structured Streaming, Parquet storage, and a live Streamlit dashboard. None of that is in scope right now — but the schemas and edge cases we build in M1 directly determine how hard M2 will be.

4. Suggested Work Split

The table below assigns M1 responsibilities across five roles and flags which tasks need the most time. Adapt role ownership to your actual team size and strengths.

Role	Milestone 1 Responsibilities	Effort
Schema Owner	Lead AVRO schema design for both feeds. Define all field types, enums, nullable unions, and versioning. Lock schemas before generator runs at scale.	HIGH — schemas are the foundation everything else depends on. Wrong types or missing fields mean breaking changes later.
Generator Dev	Implement and own the Python generator: generate.py, order_feed.py, restaurant_feed.py, config.py. Build the full state machine and all edge-case injection logic.	HIGH — this is the primary coding deliverable of M1. The anomaly injection (out-of-order, duplicates, missing steps) is the trickiest part.
Data Validator	Run the generator and inspect sample outputs in both JSON and AVRO. Verify edge cases are present, event_time values are correct, and join keys appear in both feeds.	MEDIUM — critical before submission. One bad schema or missing field is easy to miss without a dedicated validation pass.
Docs Lead	Own the main README.md. Write the design justification (why these feeds, what analytics they enable, how schemas support event-time processing). Polish all documentation.	MEDIUM-HIGH — the grader reads the docs first. The justification section (1.1) is explicitly graded and must be clear and complete.
Repo Owner	Create and maintain the private GitHub repository. Set up folder structure, enforce commit conventions, ensure all deliverables are pushed and accessible before the deadline.	LOW-MEDIUM — mostly setup and coordination, but a disorganised repo can cost marks on presentation.

M1 Priority Order — What to Do First

These are the four M1 tasks ranked by how much they block everything else. Do them in this order.

#	Task	Why This Order
1	Lock the AVRO schemas	The generator is written against the schemas. If schemas change after the generator is coded, all output breaks. Schema Owner and Generator Dev must agree on every field before any Python code is written.
2	Build and test the generator core	Normal orders first — get the state machine (PLACED to DELIVERED) working cleanly before adding any anomaly injection. Anomalies on top of broken base logic are impossible to debug.
3	Add and verify edge-case injection	Out-of-order, duplicates, missing steps, impossible durations, courier offline. Each one needs to be verified in the output JSON — not just assumed to work. Data Validator owns this check.

4	Write the README design justification	This is explicitly graded in sub-section 1.1. It cannot be written well as a last-minute afterthought — it requires understanding of the schema choices and analytics intentions. Write it while the design decisions are still fresh.
---	--	--

5. Repository Structure

Everything lives in a single private GitHub repository. The layout below is already implemented for Milestone 1.

Path	Contents
README.md	Project overview, team structure, design notes, analytics roadmap, assumptions
schemas/order_lifecycle.avsc	AVRO schema — Order Lifecycle Events (18 fields)
schemas/restaurant_events.avsc	AVRO schema — Restaurant Events (17 fields)
generator/generate.py	Main CLI entry point — runs the full simulation
generator/config.py	All tunable parameters (rates, distributions, zone weights)
generator/feeds/order_feed.py	Order Lifecycle event generator + anomaly injection
generator/feeds/restaurant_feed.py	Restaurant Events generator + capacity signals
generator/README.md	Generator usage docs, CLI reference, edge-case table
sample_data/*.json	481 order events + 396 restaurant events (pretty-printed JSON)
sample_data/*.avro	Same records in binary AVRO container format

6. Immediate Next Steps for M1

Use this as a checklist in your first team meeting. Everything here should be completed before the M1 submission deadline.

Action	Owner
--------	-------

☐	Create the private GitHub repo and push all files from this briefing package	Repo Owner
☐	Fill in team member names and role assignments in README.md	Docs Lead
☐	Everyone clones the repo, runs: <code>pip install -r requirements.txt</code> , then <code>python generate.py</code> — confirm it works locally	Everyone
☐	Schema Owner and Generator Dev review both <code>.avsc</code> files together and agree all fields are correct before coding begins	Schema Owner + Generator Dev
☐	Generator Dev builds the order state machine (normal happy-path orders only, no anomalies yet)	Generator Dev
☐	Data Validator inspects JSON output and confirms <code>event_time</code> , <code>order_id</code> , and <code>zone_id</code> are present and correct	Data Validator
☐	Generator Dev adds all five edge cases; Data Validator confirms each anomaly type appears in the output	Generator Dev + Data Validator
☐	Docs Lead writes the feed justification section of README (why these feeds, what analytics, how schemas support event-time)	Docs Lead
☐	Final review: all four deliverables present in repo (README, schemas/, generator/, sample_data/)	Repo Owner + Docs Lead

Current M1 status: Schemas, Python generator, and sample data (JSON + AVRO) are already implemented and working. Remaining tasks: push to GitHub, assign team names, and make sure the README design justification reflects your team's agreed choices.